

Exploratory Data Analysis and Feature Engineering on Fraud Patterns

Zidio Development

Week One Report

Explanation And Findings

Abstract. *This report details the foundational steps taken to prepare and analyze transaction data for fraud detection modeling. It covers the exploratory data analysis (EDA) used to establish baselines, the extraction of time-based features to identify cyclical fraud patterns, and the final data scaling and splitting processes required to create a robust, mathematically balanced dataset for machine learning algorithms.*

1. Exploratory Data Analysis (EDA) on Fraud Patterns

The initial phase of our analysis focused on understanding the raw characteristics of the dataset. Before feeding data into a machine learning model, it is crucial to identify linear relationships between numerical features and our target variable (fraud). We implemented a correlation heatmap to expose these underlying mathematical relationships. The motive behind this was to filter out irrelevant data and avoid multicollinearity (where features are too similar to each other). As seen in the resulting heatmap, it clearly highlights which features provide the strongest predictive signals.

Beyond purely numerical data, we needed to understand how categorical elements, specifically merchant categories, influence fraud risk. We aggregated the transaction data to calculate the average fraud rate for each merchant category. The motive was to see if fraudsters target specific types of businesses more frequently than others. The output graph successfully revealed that categories like online shopping and grocery point-of-sale systems carry a significantly higher baseline risk.

1.1. Transaction Amount Distribution

Another major behavioral indicator of fraud is the transaction amount itself. In real-world scenarios, fraudsters often exhibit distinct spending behaviors compared to legitimate users they might either make tiny "test" transactions to see if a stolen card works, or they might attempt to drain an account with unusually large purchases. To investigate this, we implemented a comparative boxplot to visualize the distribution of transaction values for both legitimate (Class 0) and fraudulent (Class 1) activities.

The primary challenge in visualizing financial data is the presence of extreme outliers (e.g., occasional legitimate purchases of \$10,000+), which can squash the visual representation of the core data, making it unreadable. To solve this, we intentionally limited the y-axis of our boxplot to \$1000.

Insight Discovered: The resulting visualization is highly revealing. It clearly demonstrates a noticeable difference in the median values and the interquartile ranges

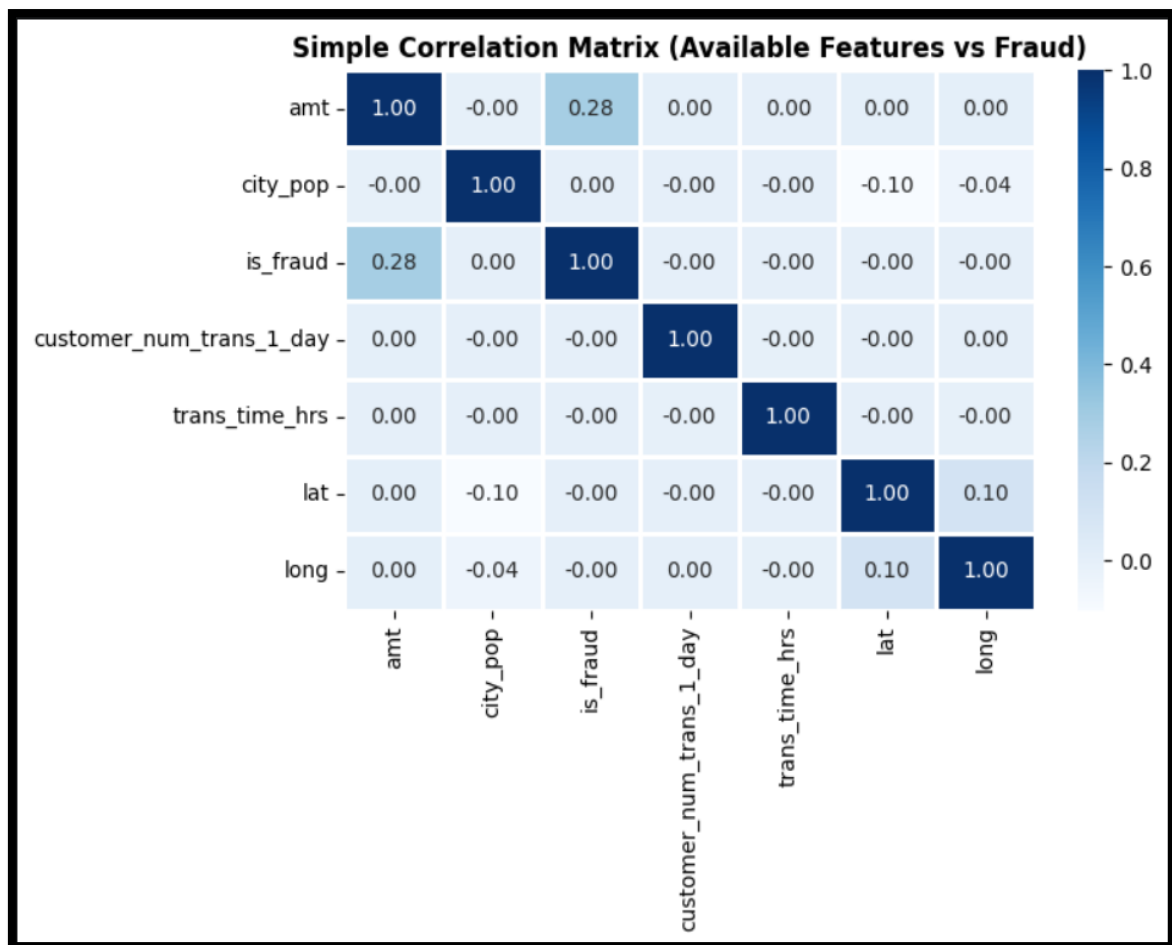


Figure 1. Feature Correlation Heatmap

between the two classes. Fraudulent transactions tend to have a higher median and a much wider, more erratic spread across the \$100 to \$1000 range compared to the tightly clustered legitimate transactions. This mathematically proves that the raw "transaction amount" is a highly valid and strong predictive feature for our upcoming machine learning model.

2. Time-Based Analysis & Feature Engineering

2.1. 24-Hour Fraud Rate Trends

Fraud is rarely a random occurrence; it is deeply tied to human behavioral cycles, particularly the time of day. However, machine learning algorithms cannot natively interpret standard raw timestamp strings. To bridge this gap, we performed feature extraction, pulling the specific "hour" from each transaction timestamp.

The motive behind this was to pinpoint temporal vulnerabilities in the banking system. By aggregating the fraud occurrences by hour, we plotted a 24-hour trend line.

Insight Discovered: The resulting graph provided a striking and actionable in-sight: fraud rates do not remain flat throughout the day. Instead, they spike significantly during late-night and early-morning hours. This makes logical sense, as legitimate users are typically asleep, leaving accounts vulnerable to automated scripts or international bad actors. Based on this precise visual evidence, we confidently engineered a specialized Boolean feature that flags any transaction occurring between 10 PM and 5 AM.

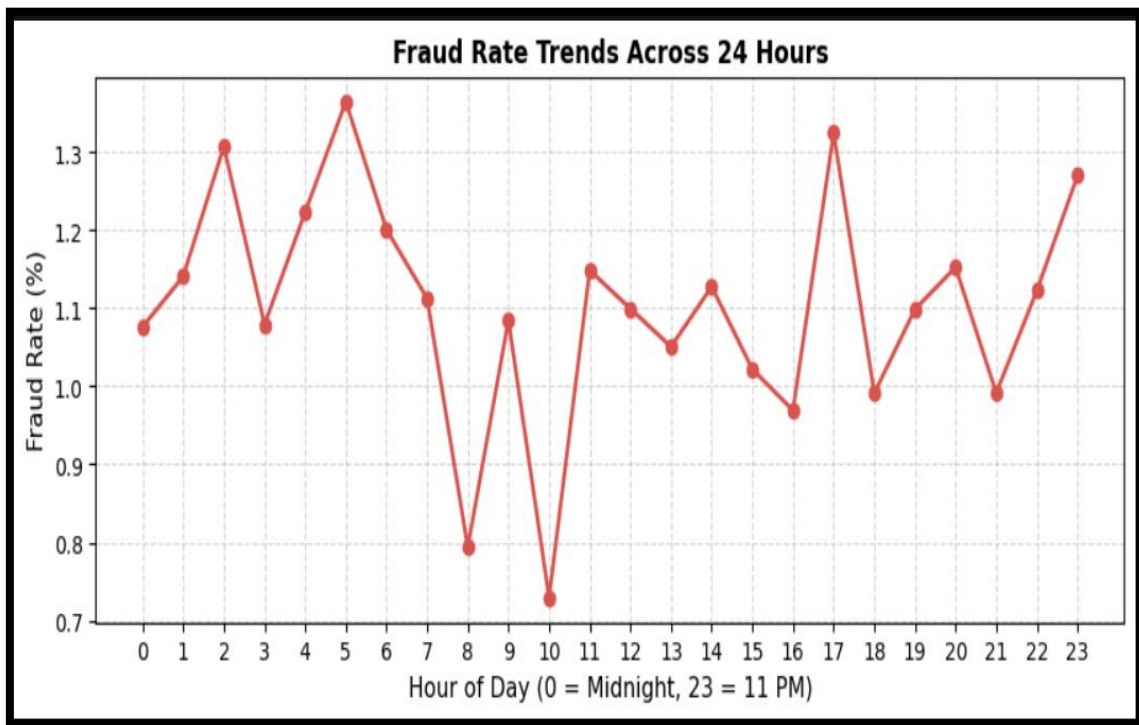


Figure 2. 24-Hour Fraud Rate Trends

2.2 Category Risk Encoding

Having established that different merchant categories carry different levels of risk, we faced a technical challenge: how do we make text-based merchant labels understandable to a mathematical ML model?

Traditional methods like One-Hot Encoding would have created a separate binary column for every single merchant category, severely bloating the dataset. Instead, we chose to implement Target Encoding (Risk Encoding).

The motive was to replace the qualitative text label with its actual historical risk probability. We mapped the previously calculated average fraud rates directly back to the main dataset.

Insight Discovered: The output data preview below validates this successful transformation. A transaction at a high-risk merchant now natively carries a higher decimal value in the `category_risk_score` column, while low-risk categories carry lower values. We have successfully transformed a meaningless text string into a highly correlated numerical feature.

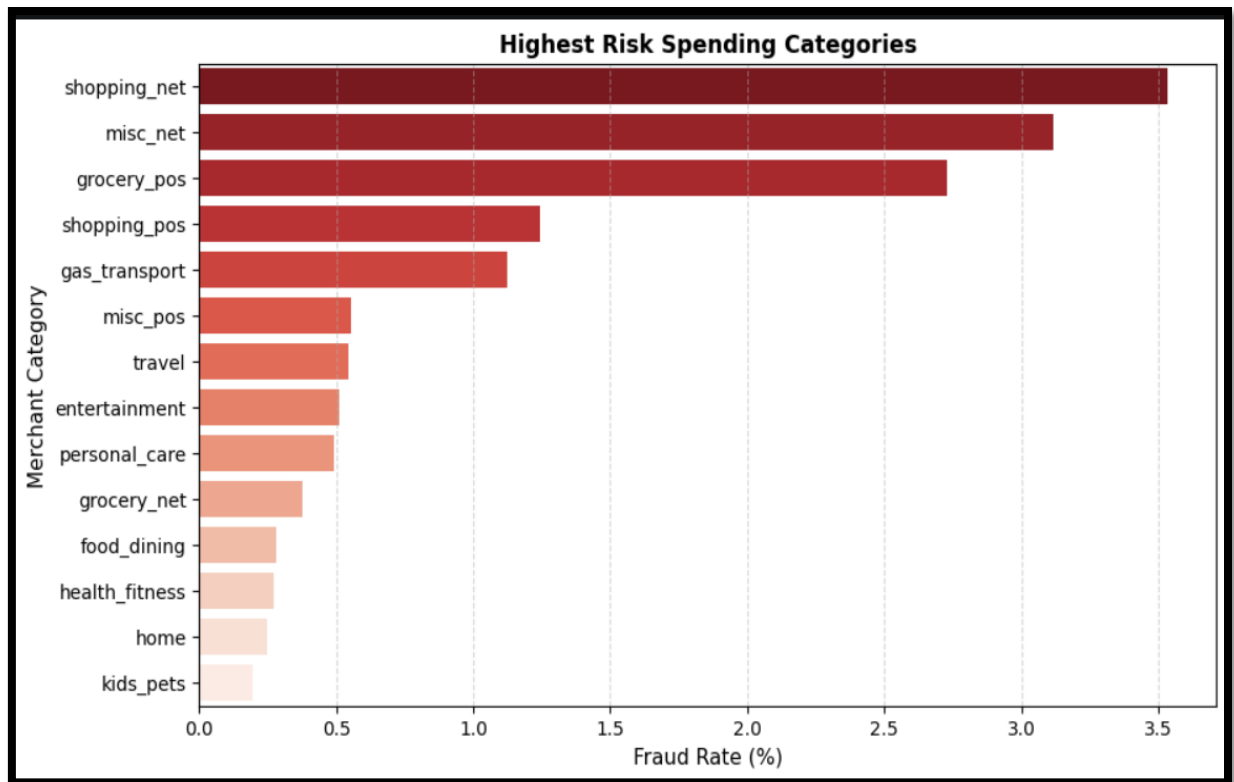


Figure 3. Category Risk Encoding Output

3. Data Splitting & Feature Scaling

3.1. Original vs. Scaled Transaction Amounts

Before moving into the actual machine learning phase, we encountered a fundamental mathematical problem in our dataset: drastic differences in numerical ranges. For instance, transaction amounts can range from \$1 to well over \$10,000. In contrast, the 'Hour of Day' feature is strictly confined between 0 and 23.

The primary motive for this step was to prevent algorithmic confusion. Distance-based or gradient-descent machine learning algorithms treat all numbers literally. If we fed the raw data into an algorithm, it would incorrectly assume that a transaction amount of 5000 is infinitely more important than an hour value of 2, simply because 5000 is a larger number. To fix this, we implemented Scikit-Learn's `StandardScaler`.

Our exact objective was to compress all input variables onto the exact same play-ing field using Z-Score normalization. This mathematically centers the data around a mean of 0 and scales it based on standard deviations.

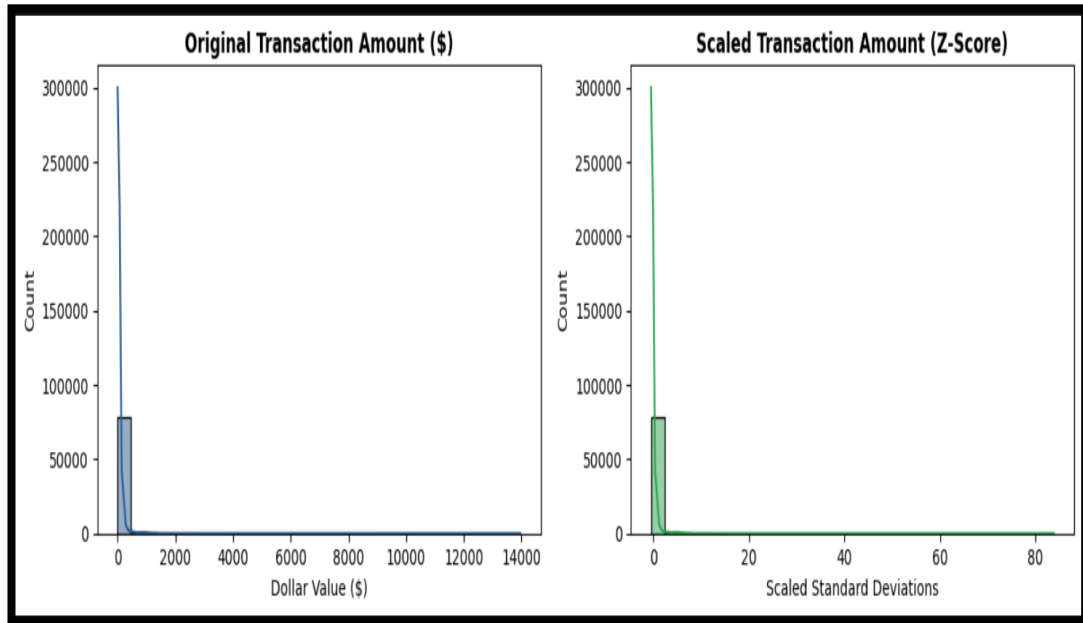


Figure 4. Transaction Amount Distribution (Boxplot)

Insight Discovered: The success of this implementation is clearly visible in the side-by-side histograms below. On the left, we see the original transaction amounts heavily skewed on a scale of \$0 to \$14,000. On the right, the Scaled Transaction Amount chart proves our success. While the absolute numbers on the X-axis have completely changed (now ranging roughly from 0 to 80 scaled standard deviations), the overall physical shape, density, and distribution of the graph are completely identical. We successfully removed the massive number bias without losing a single drop of real-world behavioral data. The features are now fully balanced for model training.

4. Conclusion

The exploratory and preparatory phases of this project have successfully transformed raw, unstructured transaction logs into a highly optimized, model-ready format. By deeply analyzing the data to find linear correlations, we identified critical vulnerabilities in specific merchant categories and time-based behavioral patterns. We addressed the limitations of machine learning algorithms by replacing raw text with calculated risk scores and by strictly standardizing massive numerical ranges into balanced Z-scores. The resulting dataset is not only mathematically sound but thoroughly cleansed of scale-based bias. With the data now properly split and stratified, the foundation is completely set for deploying high-accuracy fraud detection models in the next phase of the project.

